Universidad Tecnológica de la Mixteca

Instituto de Computación



Colección Canónica LR(0)

Informe de Actividades — Equipo 1

INTEGRANTES DEL EQUIPO

- **Lujan De la Rosa Cristian Emir**
- **Cortez Vega Jaime Fabian**
- **Carrasco Pacheco Levi Josue**
- **Nicolas Vasquez Joel Osmar**
- **Vasquez Moreno Zair De Jesus**
- **Arias Mendoza Angel Adrian**

Prof. David Martínez Torres

Compiladores • 502-A • Octubre 2026

Índice

Análisis **3**

Modelo del programa principal **5**

Asignación de actividades **8**

Clases y métodos implementados por cada subequipo **10**

Avances **14**

Conclusiones **15**

Referencias **16**

Análisis

La **colección canónica de conjuntos de ítems LR(0)** es la estructura central sobre la que se construye un analizador sintáctico ascendente determinista. Un *ítem LR(0)* de la forma $A \to \alpha \bullet \beta$ representa un estado parcial del reconocimiento: α ya ha sido desplazado a la pila y se espera ver los símbolos de β. Cuando el punto $\bullet$ llega al final ($A \to \alpha \bullet$), el ítem indica que la regla $A \to \alpha$ puede reducirse. El autómata LR(0) utiliza estos ítems para tomar decisiones de *desplazamiento* (shift) o *reducción* (reduce) sin necesidad de retroceso.

Gramática aumentada

Antes de construir la colección se **aumenta** la gramática original: se introduce un nuevo símbolo inicial S' con la producción única $S' \to S$. Esto garantiza que el estado de aceptación sea único (el que contiene $S' \to S \bullet$) y que el símbolo inicial original no aparezca a la derecha de ninguna producción, lo que simplifica los algoritmos de reducción.

Algoritmo de Clausura

Dado un conjunto de ítems I, CERRADURA(I) se obtiene aplicando iterativamente la siguiente regla hasta alcanzar el punto fijo:

1. Todo ítem de I pertenece a CERRADURA(I).
2. Si $A \to \alpha \bullet B\,\beta \in$ CERRADURA(I) y $B \to \gamma$ es una producción de la gramática, entonces $B \to \bullet\,\gamma$ se incorpora a CERRADURA(I).

La clausura expande el horizonte del autómata: al predecir que puede venir B, se agregan todos los modos posibles de empezar a reconocer B. El punto fijo se alcanza cuando no se puede añadir ningún ítem nuevo.

Algoritmo Ir_a

La función IR_A(I, X), donde I es un conjunto de ítems y X un símbolo gramatical, calcula el estado destino al consumir X desde I:

1. Se forma $J = \{\, A \to \alpha\, X \bullet \beta \mid A \to \alpha \bullet X\,\beta \in \text{CERRADURA}(I) \,\}$.
2. El resultado es CERRADURA(J).

IR_A modela la transición del autómata al leer X: avanza el punto en todos los ítems que esperaban X y clausura el resultado para capturar las nuevas predicciones.

Construcción de la colección canónica

1. El estado inicial es $I_0 = \text{CERRADURA}(\{S' \to \bullet\, S\})$.
2. Para cada estado I_k de la colección y cada símbolo X tal que $\text{IR_A}(I_k, X) \neq \emptyset$:
   - Si el estado resultante no existe aún, se añade a la colección y se registra la transición $I_k \xrightarrow{X} I_{\text{nuevo}}$.
   - Si ya existe, solo se registra la transición.
3. Se repite hasta que no se generen estados nuevos.
4. El estado que contiene $S' \to S\,\bullet$ es el estado de **aceptación**.

Detección de conflictos

Un autómata LR(0) puede presentar dos tipos de conflictos:

- **Conflicto desplazamiento/reducción**: un estado contiene tanto un ítem con el punto antes de un terminal (desplazar) como un ítem con el punto al final (reducir).
- **Conflicto reducción/reducción**: un estado contiene dos o más ítems con el punto al final, correspondientes a producciones distintas.

Una gramática libre de conflictos LR(0) produce tablas de análisis completamente deterministas. Para gramáticas con conflictos se requieren técnicas más potentes como SLR(1), LALR(1) o LR(1), que incorporan información de lookahead.

El sistema implementa los tres algoritmos en Python mediante una arquitectura modular organizada en tres capas: *modelos*, *persistencia* y *presentación*. La capa de modelos define las estructuras de datos inmutables; la capa de persistencia agrupa el lector de gramáticas y los algoritmos de clausura, ir_a y construcción de la colección; y la capa de presentación contiene la interfaz gráfica construida con Tkinter.

Modelo del programa principal

El sistema se estructura en tres capas bien diferenciadas que separan las responsabilidades de cada módulo, facilitan el mantenimiento y permiten verificar cada parte de forma independiente antes de la integración.

- **Capa de modelos** (`modelos/modelos_lr0.py`). Define cinco estructuras de datos inmutables implementadas como *dataclasses* con `frozen=True`, lo que las hace hashables y seguras para usarse como claves en conjuntos y diccionarios: `Production` (una regla de producción con validaciones en `__post_init__`), `Grammar` (gramática libre de contexto con verificación de invariantes y el método `augmented()` para generar $S' \to S$ de forma única), `LR0Item` (ítem LR(0) identificado por índice de producción y posición del punto), `LR0State` (estado del autómata con sus ítems, transiciones y la bandera `accepting`) y `CanonicalCollection` (resultado final que agrupa estados, gramática aumentada y la traza de construcción).
- **Capa de persistencia** (`persistencia/`). Agrupa tres módulos con responsabilidades distintas: `lector_gramatica.py` parsea el archivo `.txt` en dos formatos admitidos (etiquetado y posicional), infiere o valida los componentes de la gramática y construye el objeto `Grammar`; `algoritmos_lr0.py` implementa de forma directa las funciones `closure`, `goto` y `construir_coleccion`, generando además una traza detallada de cada operación realizada durante la construcción; y `algoritmo_lr0.py` actúa como *fachada* pública que re-exporta las funciones relevantes para desacoplar la interfaz de los detalles internos del módulo algorítmico.
- **Capa de presentación** (`presentacion/` e `interfaz.py`). Construida con Tkinter y la extensión `ttk`: `estilos.py` centraliza la paleta de colores y los temas globales; `tabla_estados.py` implementa el widget `TablaEstados` con scroll vertical basado en `Canvas` para mostrar cada estado I_k con sus ítems y transiciones; `ventana_resultados.py` define `VentanaResultados`, una ventana secundaria con una tabla de dos columnas (*Operación* / *Resultado*) que presenta la traza completa de construcción; e `interfaz.py` contiene la clase `InterfazColeccion`, punto de entrada de la aplicación, que orquesta la carga de gramáticas, la invocación del algoritmo y la apertura de resultados.

La arquitectura sigue el patrón **Modelo–Vista–Controlador**: la capa de modelos representa el estado del dominio, la capa de persistencia actúa como controlador que transforma datos y ejecuta la lógica, y la capa de presentación opera como vista que renderiza los resultados sin conocer los detalles algorítmicos. El uso de estructuras inmutables en la capa de modelos elimina la posibilidad de efectos secundarios entre módulos y simplifica el razonamiento sobre la corrección de los algoritmos.

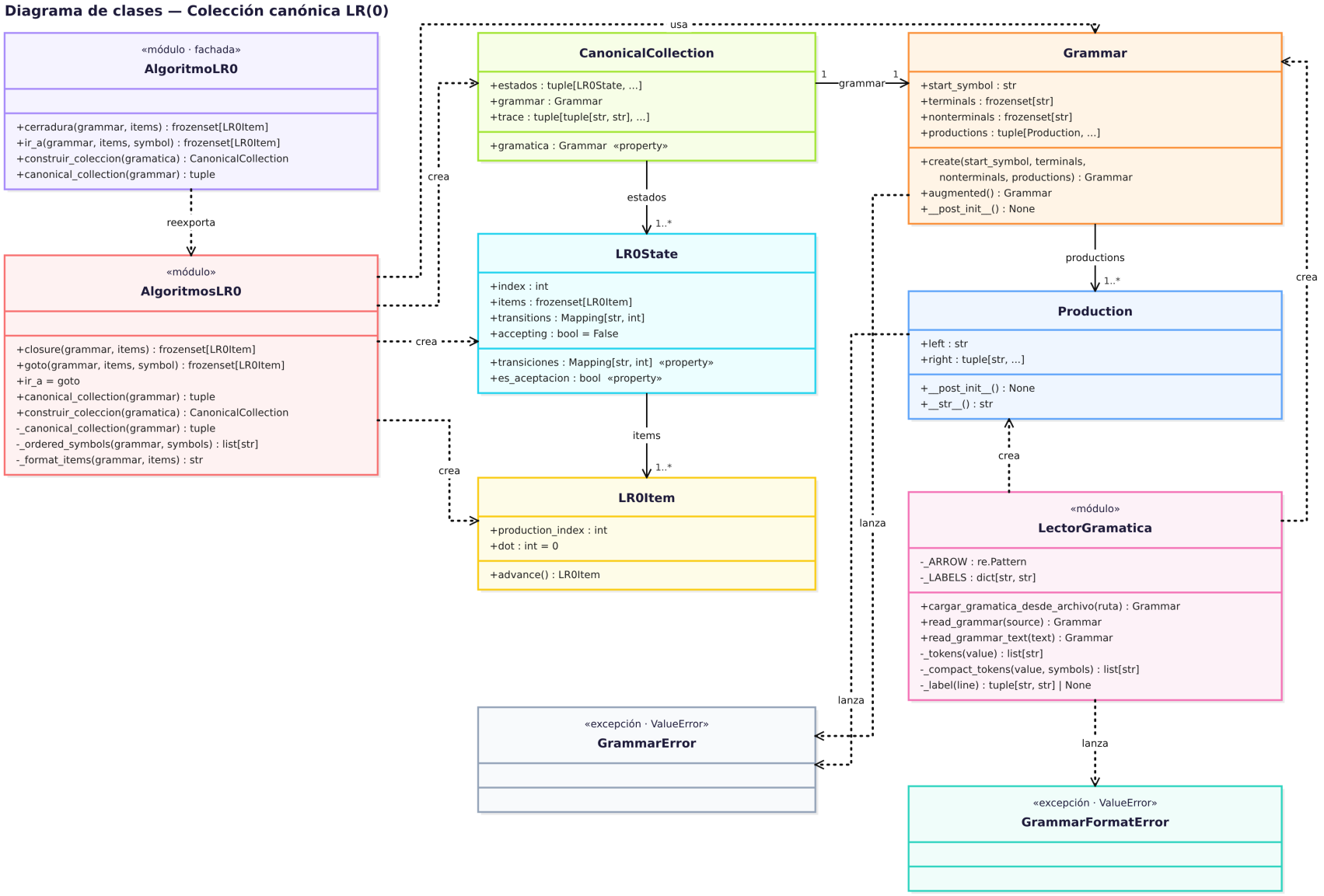


Figura 1: Diagrama de clases del sistema Colección Canónica LR(0).

Asignación de actividades

Sub equipos:

1. Cortez Vega Jaime Fabian y Lujan De la Rosa Cristian Emir
2. Arias Mendoza Angel Adrian y Vasquez Moreno Zair De Jesus
3. Carrasco Pacheco Levi Josue y Nicolas Vasquez Joel Osmar

Equipo	Actividad	Entregas clave	Período
Todos	Análisis de requerimientos, conformación de sub equipos, diseño de la arquitectura general y definición de las estructuras de datos del sistema.	Diagrama de clases preliminar y acuerdo sobre las interfaces entre capas.	Lunes 12/10.
1	Implementación de los algoritmos de clausura (`closure`), ir_a (`goto`) y construcción de la colección canónica con generación de traza.	Módulos `algoritmos_lr0.py` y `algoritmo_lr0.py` funcionales y verificados con las seis gramáticas de ejemplo.	Mar.–Vie. 13–16/10.
2	Implementación de las estructuras de datos (`Production`, `Grammar`, `LR0Item`, `LR0State`, `CanonicalCollection`) y del lector de gramáticas en sus dos formatos.	Módulos `modelos_lr0.py` y `lector_gramatica.py` con validaciones completas y soporte para ε.	Mar.–Vie. 13–16/10.
3	Diseño e implementación de la interfaz gráfica: ventana principal, componente de visualización de estados con scroll y ventana de traza de construcción.	Módulos de presentación integrados en `interfaz.py`; interfaz funcional probada en Windows y macOS.	Mar.–Vie. 13–16/10.

- El lunes 12 se dedicó íntegramente al arranque común: análisis de requerimientos, conformación de sub equipos y diseño de la arquitectura.

- Del martes 13 al viernes 16 cada sub equipo desarrolló en paralelo su módulo asignado, con reuniones diarias de sincronización para resolver dependencias.
- Los últimos dos días se reservaron para pruebas de integración, corrección de errores y redacción del informe de actividades.

Clases y métodos implementados por cada subequipo

Cortez Vega y Lujan De la Rosa:

1. **Módulo Algoritmos LR(0)** (`algoritmos_lr0.py`):
   a. **`closure(grammar, items)`** → `frozenset[LR0Item]`: calcula la clausura de un conjunto de ítems mediante iteración de punto fijo. Para cada ítem cuyo punto precede a un no terminal B, recorre todas las producciones de B en `grammar.productions` y agrega los ítems $B \to \bullet\,\gamma$ que aún no estén en el conjunto resultado. El ciclo continúa hasta que ningún ítem nuevo se incorpore.
   b. **`goto(grammar, items, symbol)`** → `frozenset[LR0Item]`: calcula el estado destino al leer `symbol` desde `items`. Primero aplica `closure` al conjunto recibido; luego selecciona los ítems cuyo punto precede exactamente a `symbol`, avanza el punto con `item.advance()` y aplica clausura al resultado. Si no hay ítems que puedan leer `symbol`, retorna el conjunto vacío. La función también se exporta con el alias `ir_a`.
   c. **`construir_coleccion(gramatica)`** → `CanonicalCollection`: función principal que orquesta la construcción completa. Primero obtiene la gramática aumentada con `gramatica.augmented()`. Inicializa I_0 como la clausura del ítem $\{S' \to \bullet\, S\}$ y lo registra en una lista de estados conocidos. Mediante un cursor BFS recorre cada estado, calcula los símbolos sobre los que existe transición (todos los símbolos inmediatamente a la derecha de algún punto), invoca `goto` para cada uno y, si el resultado es un estado nuevo, lo añade a la colección. Cada operación queda registrada en la lista `trace` como pares *(nombre_operación, descripción_resultado)*. Finalmente construye los objetos `LR0State` con su índice, ítems, transiciones y la bandera `accepting` (cierto si contiene $S' \to S\,\bullet$) y retorna un `CanonicalCollection`.
   d. **`_format_items(grammar, items)`** → `str`: función auxiliar privada que convierte un conjunto de ítems LR(0) a una representación de cadena legible, insertando el símbolo $\bullet$ en la posición correcta del cuerpo de cada producción. Usada exclusivamente para poblar la traza.
2. **Módulo Fachada** (`algoritmo_lr0.py`): Re-exporta mediante `__all__` las funciones `cerradura` (alias de `closure`), `ir_a` (alias de `goto`), `canonical_collection` y `construir_coleccion`. Este módulo aplica el patrón *fachada* para que la capa de presentación no dependa directamente de los nombres internos del módulo algorítmico.

Arias Mendoza y Vasquez Moreno:

1. **Módulo de Modelos** (`modelos_lr0.py`):
   a. **`Production`** (`frozen=True, slots=True`): *dataclass* con atributos `left: str` (cabeza de la producción) y `right: tuple[str, ...]` (cuerpo). El método

`__post_init__` valida que ni `left` ni ningún elemento de `right` sea vacío o contenga solo espacios, lanzando `GrammarError` en caso contrario. El método `__str__` retorna la producción formateada con flecha o con ε cuando `right` es la tupla vacía.

b. **`Grammar`** (`frozen=True, slots=True`): agrupa `start_symbol: str`, `terminals: frozenset[str]`, `nonterminals: frozenset[str]` y `productions: tuple[Production, ...]`. `__post_init__` verifica seis invariantes: el símbolo inicial es un no terminal, terminales y no terminales son disjuntos, existe al menos una producción, todas las cabezas de producción son no terminales, y todos los símbolos del cuerpo de alguna producción están declarados. El método de clase `create()` normaliza espacios en blanco antes de construir los `frozenset`. El método `augmented()` genera la gramática aumentada: crea un nombre único para S' (añadiendo apóstrofes mientras el nombre colisione con algún símbolo existente) y antepone la producción $S' \to S$ a las producciones originales.

c. **`LR0Item`** (`frozen=True, order=True, slots=True`): par `(production_index: int, dot: int = 0)` donde `dot` indica la posición del punto. `order=True` habilita comparación total para poder ordenar los ítems al formatear la traza. `advance()` retorna un nuevo `LR0Item` con `dot + 1`, sin modificar el objeto original (inmutabilidad garantizada por `frozen`).

d. **`LR0State`** (`frozen=True, slots=True`): representa un estado del autómata con `index: int`, `items: frozenset[LR0Item]`, `transitions: Mapping[str, int]` (diccionario símbolo → índice de estado destino) y `accepting: bool`. Las propiedades `transiciones` y `es_aceptacion` son alias en español para compatibilidad con la capa de presentación.

e. **`CanonicalCollection`** (`frozen=True, slots=True`): contenedor del resultado final con `estados: tuple[LR0State, ...]`, `grammar: Grammar` (ya aumentada) y `trace: tuple[tuple[str, str], ...]` (lista de pares operación/resultado generada por `construir_coleccion`). La propiedad `gramatica` es alias de `grammar`.

2. **Módulo Lector de Gramática** (`lector_gramatica.py`):

a. **`read_grammar_text(text)`** → `Grammar`: parsea una cadena de texto con la gramática. Recorre las líneas ignorando comentarios (`#`) y líneas vacías. Detecta automáticamente el formato: *etiquetado* (con directivas `inicio:`, `terminales:`, `no_terminales:` y `producciones:`) o *posicional* (primeras dos líneas contienen no terminales y terminales respectivamente; el símbolo inicial se infiere como el primer no terminal). Para el cuerpo de cada alternativa usa `_compact_tokens` que, cuando no hay espacios, intenta separar los símbolos por coincidencia greedy más larga, lo que permite cuerpos compactos. Lanza `GrammarFormatError` ante cualquier malformación.

b. **`cargar_gramatica_desde_archivo(ruta)`** → `Grammar`: recibe una ruta de ar-

chivo (`str` o `Path`), lee su contenido en UTF-8 y delega en `read_grammar_text`. Propaga `GrammarFormatError` en caso de error de E/S o de formato.

Carrasco Pacheco y Nicolas Vasquez:

1. **Módulo de Estilos** (`estilos.py`): Define el diccionario `COLORES` con la paleta completa del proyecto. `configurar_estilos(ventana)` recibe la ventana raíz y aplica los temas `ttk` globalmente: botón estándar (`TButton`), botón de acción principal (`Accent.TButton` con color primario y fuente en negrita), tabla (`Treeview` con encabezados resaltados), y pestañas (`TNotebook.Tab` con estado activo/inactivo diferenciado). `crear_superficie(padre)` → `tk.Frame`: función de fábrica que retorna un `Frame` con borde uniforme (`highlightthickness=1`) y fondo de superficie, usado como contenedor visual en todas las secciones.

2. **Módulo Tabla de Estados** (`tabla_estados.py`): Implementa la clase `TablaEstados(tk.Frame)` para visualizar la colección canónica. Recibe la lista de `estados` y la `gramatica` aumentada en su constructor. En `_crear()` construye un encabezado con el número total de estados, y a continuación usa un `Canvas` con barra de scroll vertical para mostrar cada estado en su propio marco: el título `I`k (en verde y con la etiqueta ńACEPTACIÓNż si `es_aceptacion` es verdadero), los ítems LR(0) formateados con el punto • en fuente monoespaciada, y la línea de transiciones en la forma `símbolo ->I`n. Permite navegar colecciones con decenas de estados sin desbordar la ventana.

3. **Módulo Ventana de Resultados** (`ventana_resultados.py`): Define `VentanaResultados(tk.Toplevel)`, ventana secundaria de 1180×760 px con un `Treeview` de dos columnas: *Operación* (390 px) y *Resultado* (700 px). Cada fila corresponde a un paso de la traza generada por `construir_coleccion`: apertura de estados (`cerradura(...)`), transiciones (`Ir_a(I`k`, X)`) y contenido de cada estado. Incluye barras de desplazamiento vertical y horizontal para trazar colecciones de decenas de estados.

4. **Módulo Interfaz Principal** (`interfaz.py`): Implementa `InterfazColeccion(tk.Tk)`, punto de entrada de la aplicación:

   - **`abrir_archivo()`**: abre un diálogo de selección de archivo `.txt`, llama a `cargar_gramatica_desde_archivo` y captura `OSError`, `UnicodeError` y `ValueError` mostrando el mensaje de error en un cuadro de diálogo.
   - **`_mostrar_gramatica()`**: vuelca el contenido crudo del archivo en el editor de solo lectura para que el usuario pueda inspeccionarlo.
   - **`analizar()`**: verifica que haya gramática cargada, invoca `construir_coleccion`, destruye la ventana de resultados anterior si existe y abre una nueva `VentanaResultados`. Actualiza la barra de estado con el número de estados LR(0) generados.
   - **`limpiar()`**: restablece todos los atributos de estado y limpia el editor de texto.

Avances

Fecha	Lujan De la Rosa Cristian Emir	Cortez Vega Jaime Fabian	Arias Mendoza Angel Adrian
12/10/2026	Análisis de requerimientos, conformación de sub equipos y diseño de la arquitectura general. Definición de la estructura de las clases de modelo.	Análisis de requerimientos y estudio del algoritmo de clausura LR(0). Diseño de la función iterativa de punto fijo.	Análisis de requerimientos. Diseño de las clases `Production` y `Grammar` con sus validaciones.
13/10/2026	Implementación del lector de gramáticas en formato posicional y pruebas con la Gramática 1.	Implementación de `closure` con iteración de punto fijo; primeras pruebas con la Gramática 1.	Implementación de `Production` y `Grammar` con `augmented()`; pruebas de validaciones.
14/10/2026	Extensión del lector al formato etiquetado (`inicio:`, `terminales:`). Soporte para flechas `->` y comentarios.	Implementación de `goto` e integración con `closure`. Pruebas con las Gramáticas 2 y 3.	Implementación de `LR0Item` con `advance()` y de `LR0State` con sus transiciones.
15/10/2026	Pruebas del lector con las seis gramáticas de ejemplo e integración con el módulo de modelos.	Implementación de `construir_coleccion` con generación de traza. Pruebas con las Gramáticas 4, 5 y 6.	Implementación de `CanonicalCollection`. Pruebas de integración con los algoritmos de clausura e ir_a.
16/10/2026	Pruebas finales del lector con archivos malformados; verificación de manejo de errores. Redacción del informe.	Pruebas finales y comparación de resultados con los esperados del profesor. Apoyo en el informe.	Pruebas finales del módulo de modelos. Redacción de conclusiones y revisión de la rúbrica.

Fecha	Vasquez Moreno Zair De Jesus	Nicolas Vasquez Joel Osmar	Carrasco Pacheco Levi Josue
12/10/2026	Análisis de requerimientos. Investigación manual del algoritmo LR(0) sobre gramáticas sencillas.	Análisis de requerimientos. Diseño del módulo de estilos y la paleta de colores.	Análisis de requerimientos. Diseño del componente `TablaEstados` con scroll vertical.
13/10/2026	Práctica manual de clausura e ir_a con la Gramática 1 para verificar resultados esperados.	Implementación de `estilos.py`: diccionario `COLORES` y temas `ttk` globales.	Implementación de `TablaEstados` con `Canvas` y barra de scroll vertical.
14/10/2026	Apoyo en pruebas de `goto` con las Gramáticas 2 y 3. Verificación de estados generados.	Implementación de `VentanaResultados` con tabla de traza de dos columnas y doble scroll.	Implementación de `InterfazColeccion` con botones de carga, análisis y limpieza.
15/10/2026	Pruebas del flujo completo con todas las gramáticas de ejemplo. Verificación del estado de aceptación.	Integración de `VentanaResultados` con la traza de `construir_coleccion`; corrección de formato.	Pruebas de integración de la interfaz completa en macOS. Corrección de detalles visuales en la barra de estado.
16/10/2026	Pruebas finales y verificación del número de estados con los resultados del profesor. Apoyo en el informe.	Redacción de los elementos del informe. Integración del diagrama UML en el documento.	Redacción final del documento, revisión de ortografía y verificación del PDF generado.

Conclusiones

El desarrollo del presente proyecto permitió al equipo comprender de manera rigurosa y práctica el papel que desempeña la **colección canónica de conjuntos de ítems LR(0)** en la construcción de analizadores sintácticos ascendentes. A diferencia del análisis predictivo descendente LL(1) —estudiado en la asignación anterior mediante los conjuntos PRIMERO y SIGUIENTE— el análisis LR(0) no necesita predecir la producción a aplicar: en cambio, acumula información en la pila a medida que desplaza símbolos de la entrada y decide reducir cuando el estado activo contiene un ítem completo. La colección canónica es precisamente el catálogo exhaustivo de todos los estados posibles de ese proceso.

La implementación de los algoritmos de clausura e ir_a puso de manifiesto la naturaleza composicional del autómata LR(0): `closure` expande horizontalmente un conjunto de ítems capturando todas las predicciones posibles para un no terminal, mientras que `goto` desplaza el punto a través de un símbolo y vuelve a clausurar, generando un nuevo estado. La correcta composición iterativa de ambas funciones en `construir_coleccion` garantiza que la colección sea mínima (sin estados duplicados) y completa (cubre todos los prefijos viables). El uso de `frozenset` como representación canónica de cada estado fue clave para detectar estados ya conocidos en tiempo constante.

El diseño en tres capas —modelos, persistencia y presentación— demostró su valor durante la integración: cada sub equipo pudo desarrollar y verificar su módulo de forma autónoma antes de conectarlo con los demás, ya que las interfaces entre capas estaban definidas por las *dataclasses* inmutables. El patrón fachada aplicado en `algoritmo_lr0.py` simplificó la capa de presentación al exponer solo las funciones relevantes y ocultar los detalles internos del módulo algorítmico.

El sistema de traza detallada incorporado en `construir_coleccion` resultó especialmente valioso en el contexto educativo: al registrar cada llamada a `closure` e `ir_a` con su resultado, el usuario puede seguir paso a paso la construcción del autómata y confrontarla con los cálculos manuales realizados en clase. Esta característica convierte la herramienta en un instrumento de verificación, no solo de cómputo.

Como perspectiva futura, la arquitectura del sistema facilita la extensión hacia técnicas más potentes: los conjuntos SIGUIENTE —calculados en la asignación anterior— pueden incorporarse directamente para construir tablas SLR(1), y con la adición de lookaheads por ítem se puede escalar a LALR(1) o LR(1). La modularidad ya establecida minimizaría el esfuerzo de esas extensiones.

En síntesis, el proyecto consolidó la comprensión de la fase ascendente del diseño de compiladores y reforzó buenas prácticas de ingeniería de software —estructuras inmutables, separación de responsabilidades y pruebas incrementales— que serán aplicadas en las etapas subsecuentes del curso.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.ł ed.). Pearson Educación.

Appel, A. W., y Palsberg, J. (2004). *Modern compiler implementation in Java* (2nd ed.). Cambridge University Press.

Cooper, K. D., y Torczon, L. (2012). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.

Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., y Langendoen, K. (2012). *Modern compiler design* (2nd ed.). Springer. https://doi.org/10.1007/978-1-4614-4699-6

Martínez Torres, D. (s. f.). *Tema 3. Análisis sintáctico* [Diapositivas de curso]. Instituto de Computación, Universidad Tecnológica de la Mixteca.

Mogensen, T. Æ. (2017). *Introduction to compiler design* (2nd ed.). Springer. https://doi.org/10.1007/978-3-319-66966-3

Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.